



Análise Léxica: Tokens, Expressões Regulares e Autômatos

Prof. Dr. Antonio Marcos A. Ferreira

Aula 03 - 13/03/2026

Compiladores

Objetivos da Aula

- Compreender o papel da Análise Léxica em um compilador.
- Definir *tokens*, *lexemas* e padrões.
- Relacionar expressões regulares com reconhecimento de tokens.
- Estudar autômatos (AFN e AFD) e conversão entre eles.
- Conectar a teoria ao projeto do mini-compilador (lexer).

Papel da Análise Léxica

- Primeira fase do compilador (*front-end*), responsável por iniciar o processo de tradução.
- Converte o fluxo de caracteres do código-fonte em unidades significativas chamadas *tokens*.
- Fornece uma entrada estruturada e simplificada para o analisador sintático (*parser*).
- Identifica e reporta erros léxicos, como caracteres inválidos ou malformados.
- Remove espaços em branco e comentários, facilitando a análise posterior.
- Atua como uma ponte entre a codificação do programador e a análise sintática da linguagem.

Definições Básicas

- 1) **Token**: unidade léxica significativa reconhecida pelo compilador.
 - Exemplos: palavra-chave (if, while), identificador (variavel, total), operador (+, =), delimitador (;, {}).
- 2) **Lexema**: sequência exata de caracteres do código-fonte que se enquadra em um *token*.
 - Exemplo: na instrução `int soma = a + b;`, os lexemas são **int**, **soma**, **=**, **a**, **+**, **b**, **;**.
- 3) **Padrão**: a regra formal (geralmente expressões regulares) que descreve como formar os *tokens*.
 - Exemplos de padrões:
 - Identificadores $\rightarrow [a-zA-Z_][a-zA-Z0-9_]*$
 - Números inteiros $\rightarrow [0-9]^+$

Expressões Regulares

São notações formais que descrevem padrões de cadeias de caracteres, usadas para especificar *tokens* em compiladores.

- Exemplos práticos:
 - $[0-9]^+$ → números inteiros (123, 45, 7)
 - $[a-zA-Z_][a-zA-Z0-9_]^*$ → identificadores (soma, total1, _variavel)
 - `if|else|while` → palavras-chave específicas
- Uso em compiladores:
 - Permitem a geração automática de analisadores léxicos.
 - Fundamentam ferramentas como Lex/Flex, ANTLR e JFlex.
- Importância:
 - Base para construir autômatos finitos (AFN/AFD).
 - Facilitam a detecção rápida e eficiente de *tokens* durante a compilação.

Autômatos Finitos

- **AFN (Autômato Finito Não-Determinístico):** Um modelo computacional que tem um número finito de estados e transita entre eles com base nas entradas recebidas.
 - Mais intuitivo e fácil de construir a partir de uma **Expressão Regular (ER)**.
 - Permite múltiplas transições possíveis ou até transições vazias (ϵ).
 - Usado como modelo teórico para derivar o AFD.
- **AFD (Autômato Finito Determinístico):** Um modelo matemático que reconhece sequências de símbolos (linguagens) e para cada entrada determina um único caminho a ser seguido.
 - Para cada estado e símbolo de entrada existe apenas uma transição possível.
 - É a forma usada na prática para implementar o analisador léxico.
- Equivalência: AFN e AFD reconhecem exatamente a mesma classe de linguagens regulares.

Autômatos Finitos

- Etapas de Construção do *Lexer*:
 - 1) $ER \rightarrow AFN$ (Construção de *Thompson*)
 - pega cada ER de *token* e monta um AFN correspondente;
 - 2) $AFN \rightarrow AFD$ (Construção de subconjuntos)
 - transformar o AFN em um AFD, que será usado na prática pelo analisador léxico
 - 3) AFD Minimizado $\rightarrow$ mais eficiente para implementação.
 - é a versão otimizada, com o menor número possível de estados que ainda reconhece exatamente a mesma linguagem.

Exemplo Prático de Análise Léxica

Tabela 1: *Tokens* reconhecidos.

Lexema	Token	Padrão
int	keyword	reservado
soma	id	[a-zA-Z_][a-zA-Z0-9_]*
=	atribuição	=
a	id	[a-zA-Z_][a-zA-Z0-9_]*
+	operador	+
b	id	[a-zA-Z_][a-zA-Z0-9_]*
;	delimitador	;

1) Código-fonte (entrada)

```
int soma = a + b ;
```

2) Fluxo de caracteres

```
int soma = a + b ;
```

3) *Tokens* Reconhecidos

4) Saída do Analisador Léxico

```
<keyword,int> <id,soma> <atribuição,=> <id,a> <op,+>
```

```
<id,b> <delimitador,;>
```


Exemplo com Flex

Arquivo: lexer.l

```
%{
#include <stdio.h>
%}

%%

"int"      { printf("<keyword,int> "); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("<id,%s> ", yytext); }
"="        { printf("<atribuição,=> "); }
"+"        { printf("<op,+> "); }
";"        { printf("<delimitador,> "); }
[ \t\n]    ; /* Ignora espaços e quebras de linha */
.          { printf("<erro,%s> ", yytext); }

%%

int main(void) {
    yylex();
    return 0;
}
```

Entrada (código-fonte)

```
int soma = a + b;
```

Saída gerada pelo lexer

```
<keyword,int> <id,soma> <atribuição,=> <id,a> <op,+>
<id,b> <delimitador,>
```

Como usar no Linux/WSL

Salve o arquivo como lexer.l.

Compile com:

```
flex lexer.l
gcc lex.yy.c -o lexer -lfl
```

Execute:

```
echo "int soma = a + b;" | ./lexer
```

Ferramentas Sugeridas

1) Flex (Fast Lexical Analyzer Generator)

- Gera analisadores léxicos em C/C++ a partir de especificações em expressões regulares. É amplamente utilizado em cursos e projetos educacionais.
- Permite ignorar espaços em branco/comentários e definir padrões para tokens declarativamente.

2) Jflex

- Gerador de analisadores léxicos para Java, com suporte a Unicode.
- A partir de regex definidas, produz autômatos determinísticos (AFD), facilitando a implementação de lexers robustos.

3) RE/flex

- Alternativa moderna ao Flex, com compatibilidade com C++, suporte a Unicode, scanners thread-safe e geração de código elegante.

Ferramentas Sugeridas

4) ANTLR (ANother Tool for Language Recognition)

- Ferramenta poderosa que une lexer e parser, gerando código em várias linguagens (Java, Python, C#, C++, Go, etc.) a partir de gramáticas em EBNF.
- Oferece suporte completo: geração de árvores sintáticas (parse trees), tratamento de erros, plugins para IDEs (IntelliJ, VS Code) e ambiente visual (ANTLR Lab).
- É mais moderna e flexível que o par clássico Flex + Bison, além de separar claramente gramática de código, promovendo melhor reaproveitamento e manutenção.

5) JavaCC (Java Compiler Compiler)

- Gerador de lexer e parser em Java, que segue gramáticas do tipo LL(k).
- Permite definir tokens por expressões regulares, oferece ferramentas adicionais como JJTree (para árvores sintáticas) e JJDoc (para documentação automática).

6) GALS

- Ferramenta educacional desenvolvida na UFSC que gera analisadores léxicos e sintáticos em Delphi, C++ e Java.
- Ideal para projetos acadêmicos e experimentação de diversas tecnologias

Conexão com o Trabalho

- Semana 3: Implementar o Analisador Léxico (lexer).
 - Definir expressões regulares para tokens da mini-linguagem.
 - Construir autômatos (AFN/AFD) correspondentes.
 - Entregar código + relatório de testes.